R PROGRAMMING
PRACTICAL EXAM MASTER GUIDE

Syllabus-aligned code • Step-by-step explanations • Outputs • Viva preparation

Prepared for: Data Analysis Using R Programming Lab (25CSE221)

Based on the uploaded lab syllabus, notebook and Data Visualization with R handout. The material below distinguishes exact syllabus tasks from safe implementation corrections and illustrative examples.

What this guide covers

- Experiment 1: vectors, lists, matrices, matrix calculations and sorting
- Experiment 2: string operations and palindrome function
- Experiments 3–4: data visualization, charts, interactive charts and export
- Notebook-linked visualization workflow with Titanic CSV
- Program outputs, code explanations, common errors and viva questions

Important: The notebook contains a data-visualization lab focused on a Titanic CSV and four charts. The official syllabus text groups several vector/list/matrix tasks under Experiment 1 and describes string programs under Experiment 2. Experiment labels in teaching material may differ; match the task statement given by your examiner.

How to use this guide

Study each experiment in this order: (1) understand the purpose, (2) run the complete code, (3) compare output and chart, (4) explain the important lines, and (5) practise viva questions. Code is written in plain R and should be run in RStudio.

Contents

- 1. Setup and common R commands
- 2. Experiment 1(a): sequence and vector filtering
- 3. Experiment 1(b): list containing vector, matrix and nested list
- 4. Experiment 1(c): matrix addition, subtraction and transpose product
- 5. Experiment 1(d): column sums, row means, total sum and column sorting
- 6. Experiment 2(a): name string operations
- 7. Experiment 2(b): is_palindrome() function
- 8. Visualization setup and CSV reading
- 9. Bar chart
- 10. Density plot (the handout labels this as a line plot)
- 11. Scatter plot
- 12. Heatmap
- 13. Interactivity with plotly
- 14. Export static and interactive charts
- 15. Troubleshooting
- 16. Viva bank and quick revision

Source alignment note: The syllabus explicitly lists vector filtering, composite lists, matrix operations/statistics/sorting, string processing, palindrome detection, and data visualization with CSV input, chart customization, interactivity and export. The notebook/handout adds specific Titanic plotting code. Where its code contained inconsistent names or labels, this guide gives runnable corrections and explains them.

1. Setup and common R commands

1.1 Install and load packages

Install packages only once on a computer. Load them each time you start a fresh R session.

```
install.packages(c("ggplot2", "dplyr", "plotly", "htmlwidgets"))

library(ggplot2)
library(dplyr)
library(plotly)
library(htmlwidgets)
```

If the packages are already installed, skip the install.packages() line. For the base-R vector, list and matrix exercises, no extra package is required.

1.2 Useful commands

Command	Purpose	Example
print(x)	Displays an object	print(x)
cat(...)	Displays combined text/values	cat("Count:", length(x), "\n")
class(x)	Reports object class	class(list(1, "a"))
length(x)	Counts elements in a vector/list	length(c(1,2,3))
nrow(x), ncol(x)	Matrix/data-frame dimensions	nrow(m)
head(df)	Shows first six rows by default	head(titanic)
str(df)	Shows structure and column types	str(titanic)
names(df)	Lists column names	names(titanic)

1.3 R practical rules

- R is case-sensitive: Survived and survived are different names.
- Indexing starts at 1, not 0.
- Use <- for assignment; use == to compare values for equality.
- A matrix has one basic data type; a list can contain different types and nested objects.
- When loading a file, check names(df) before writing code that uses column names.

2. Experiment 1(a): Sequence and vector filtering

Syllabus task: create multiples of 5 from 5 to 60 using seq(), display them, and filter values greater than 15 and less than 45.

```
x <- seq(5, 60, by = 5)
cat("Multiples of 5:", x, "\n")

filtered_x <- x[x > 15 & x < 45]
cat("Filtered values:", filtered_x, "\n")
```

Expected output

```
Multiples of 5: 5 10 15 20 25 30 35 40 45 50 55 60
Filtered values: 20 25 30 35 40
```

Line-by-line explanation

- seq(5, 60, by = 5) starts at 5, increases by 5 each time, and does not pass 60.
- x stores the resulting numeric vector.
- x > 15 creates a TRUE/FALSE result for each element.
- x < 45 creates a second condition.
- & requires both conditions to be TRUE, so 15 and 45 are excluded.
- x[condition] selects only elements for which the condition is TRUE.

Common viva

- What does seq() do? It generates a sequence of values.
- What does by = 5 mean? Add 5 between successive values.
- Why is 45 excluded? The condition is strictly less than 45.
- Difference between & and &&: & is element-wise; && tests only the first element of each logical vector, normally for if conditions.

3. Experiment 1(b): List with a vector, matrix and nested list

Syllabus task: build a list containing a vector, matrix and another list; display the class of each element; count objects in the list; and find the length of the first vector.

```
student_data <- list(
  scores = c(80, 85, 90),
  marks_matrix = matrix(1:6, nrow = 2, ncol = 3),
  details = list(name = "Yawar", semester = 3)
)

cat("Class of first element:", class(student_data[[1]]), "\n")
cat("Class of second element:", class(student_data[[2]]), "\n")
cat("Class of third element:", class(student_data[[3]]), "\n")
cat("Number of objects in list:", length(student_data), "\n")
cat("Length of first vector:", length(student_data[[1]]), "\n")
```

Expected output

```
Class of first element: numeric
Class of second element: matrix array
Class of third element: list
Number of objects in list: 3
Length of first vector: 3
```

Line-by-line explanation

- list(...) creates a list and permits different kinds of objects inside it.
- scores is a numeric vector of three elements.
- marks_matrix is a matrix of six values arranged in two rows and three columns.
- details is a nested list containing name and semester.
- student_data[[1]] extracts the actual first object, so class() reports numeric.
- student_data[[2]] extracts the matrix; in R, its class is usually c("matrix", "array").
- length(student_data) is 3 because the outer list has three elements.
- length(student_data[[1]]) is 3 because the first element is a vector of length 3.

Important: [], [[]] and $

Expression	Meaning	Type returned here
student_data[1]	Select element 1 and preserve list wrapper	list
student_data[[1]]	Extract the actual first element	numeric vector
student_data$scores	Extract by element name	numeric vector
student_data$details$name	Access name in nested list	character value

Viva

- Can a list hold a matrix? Yes.
- Why does class(matrix(...)) return matrix and array? A matrix is a two-dimensional array, so its class attribute includes both.
- What does $ do? Accesses an element by name.

4. Experiment 1(c): Matrix operations

Syllabus task: create matrices and perform addition, subtraction, and product of a matrix with its transpose.

```
A <- matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE)
B <- matrix(c(5, 6, 7, 8), nrow = 2, byrow = TRUE)

cat("Matrix A:\n"); print(A)
cat("Matrix B:\n"); print(B)

cat("A + B:\n"); print(A + B)
cat("A - B:\n"); print(A - B)
cat("A %*% t(A):\n"); print(A %*% t(A))
```

Expected output

```
Matrix A:
     [,1] [,2]
[1,]    1    2
[2,]    3    4
Matrix B:
     [,1] [,2]
[1,]    5    6
[2,]    7    8
A + B:
     [,1] [,2]
[1,]    6    8
[2,]   10   12
A - B:
     [,1] [,2]
[1,]   -4   -4
[2,]   -4   -4
A %*% t(A):
     [,1] [,2]
[1,]    5   11
[2,]   11   25
```

Explanation

- matrix(c(...), nrow = 2, byrow = TRUE) fills values row by row.
- A + B and A - B operate on matching positions. The matrices must have compatible dimensions.
- t(A) transposes A, swapping rows and columns.
- %*% is matrix multiplication. Do not confuse it with * , which multiplies matching elements position by position.
- A is 2 x 2 and t(A) is 2 x 2, so A %*% t(A) is valid and returns a 2 x 2 matrix.

Viva

- Difference between * and %*%? Element-wise multiplication versus matrix multiplication.
- What does t(A) do? Transposes the matrix.
- Can matrices of different sizes always be added? No; dimensions must match for ordinary matrix addition/subtraction.

5. Experiment 1(d): Matrix summaries and sorting

Syllabus task: calculate column sums, mean across rows, total sum, and sort matrix elements column-wise in ascending order.

```
M <- matrix(c(8, 3, 6, 1, 9, 4), nrow = 3, ncol = 2)
cat("Matrix M:\n"); print(M)

cat("Column sums:\n"); print(colSums(M))
cat("Row means:\n"); print(rowMeans(M))
cat("Total sum:", sum(M), "\n")

cat("Each column sorted ascending:\n")
M_sorted <- apply(M, 2, sort)
print(M_sorted)
```

Expected output

```
Matrix M:
     [,1] [,2]
[1,]    8    1
[2,]    3    9
[3,]    6    4
Column sums:
[1] 17 14
Row means:
[1] 4.5 6.0 5.0
Total sum: 31
Each column sorted ascending:
     [,1] [,2]
[1,]    3    1
[2,]    6    4
[3,]    8    9
```

Explanation

- colSums(M) adds values down each column: column 1 is 8 + 3 + 6 = 17; column 2 is 1 + 9 + 4 = 14.
- rowMeans(M) calculates the average across each row: (8 + 1)/2 = 4.5, (3 + 9)/2 = 6, and (6 + 4)/2 = 5.
- sum(M) adds all elements, resulting in 31.
- apply(M, 2, sort) applies sort to each column. The 2 means operate across columns; 1 would apply over rows.

Note: Some teaching notes may phrase sorting as “sort matrix elements across columns.” apply(M, 2, sort) sorts each column independently and preserves the matrix shape.

Viva

- colSums() versus rowSums()? Column totals versus row totals.
- What does apply(M, 2, sort) do? Sorts each column ascending.
- What is sum(M)? Total of every matrix element.

6. Experiment 2(a): Name string operations

Syllabus task: input a name, count its characters, convert it to uppercase, and display its middle character. The code below handles odd- and even-length strings, returning one or two middle characters as appropriate.

```
name <- readline(prompt = "Enter your name: ")
n <- nchar(name)
cat("Number of characters:", n, "\n")

upper_name <- toupper(name)
cat("Uppercase name:", upper_name, "\n")

if (n == 0) {
  cat("No middle character: empty input.\n")
} else {
  start <- (n + 1) %/% 2
  end <- (n + 2) %/% 2
  mid <- substring(name, start, end)
  cat("Middle character(s):", mid, "\n")
}
```

Sample input and output

```
Enter your name: Rahul
Number of characters: 5
Uppercase name: RAHUL
Middle character(s): h
```

Explanation

- readline() receives user input as text.
- nchar(name) counts the characters, including spaces.
- toupper(name) changes letters to uppercase.
- %/% performs integer division. start and end identify the middle position(s).
- substring(name, start, end) extracts from the start position through the end position, including both endpoints.
- For an odd length such as 5, start and end are both 3. For an even length such as 4, start is 2 and end is 3, so both middle characters are returned.

Additional string functions

Function	Use	Example output
nchar("Ya war")	Counts characters including the space	6
toupper("Yawar")	Uppercase conversion	"YAWAR"
tolower("HELLO")	Lowercase conversion	"hello"
substring("Extract", 5, 7)	Extract positions 5 to 7	"act"

7. Experiment 2(b): Palindrome function

Syllabus task: define is_palindrome() with one parameter that checks whether a string reads the same forward and backward.

```
is_palindrome <- function(str) {
  str <- tolower(str)
  chars <- strsplit(str, "")[[1]]
  reversed <- paste(rev(chars), collapse = "")

  if (str == reversed) {
    print("Palindrome")
  } else {
    print("Not a palindrome")
  }
}

is_palindrome("madam")
is_palindrome("hello")
is_palindrome("LEVEL")
```

Expected output

```
[1] "Palindrome"
[1] "Not a palindrome"
[1] "Palindrome"
```

Line-by-line explanation

- function(str) defines a function with one parameter called str.
- tolower(str) makes the comparison case-insensitive for letters.
- strsplit(str, "")[[1]] splits the string into characters and [[1]] extracts the character vector from the returned list.
- rev(chars) reverses the character vector.
- paste(..., collapse = "") joins characters without spaces.
- if (str == reversed) checks whether the original and reversed strings are equal.

Scope note: This implementation ignores letter case but does not remove spaces or punctuation. Thus “Madam” is accepted, but a phrase with spaces is not automatically normalized.

Viva

- What is a palindrome? A string that reads the same forward and backward.
- Why use tolower()? To make “MADAM” and “madam” compare equally.
- What does rev() do? Reverses the order of elements.
- What does collapse = "" do? Joins characters with no separator.

8. Visualization setup: Load the Titanic CSV

Your Data Visualization with R handout uses a Titanic dataset and the ggplot2, dplyr and plotly packages. It demonstrates bar, density/line-style, scatter and heatmap plots, then interactive rendering and file export.

8.1 Install/load and import

```
install.packages(c("ggplot2", "dplyr", "plotly", "htmlwidgets"))

library(ggplot2)
library(dplyr)
library(plotly)
library(htmlwidgets)

titanic <- read.csv(file.choose())
head(titanic)
str(titanic)
names(titanic)
```

Use file.choose() to browse for the correct CSV. Check names(titanic) and confirm the file really contains columns named Pclass, Survived, Age and Fare. Column names differ across Titanic datasets.

8.2 Prepare the survival column

```
titanic$Survived <- factor(titanic$Survived, levels = c(0, 1))
```

- The $ operator selects a named column from a data frame.
- factor() tells R to treat survival as a categorical variable.
- In most common Titanic training CSVs, Survived is coded 0 and 1. If the file uses other labels, adapt the factor levels to the actual column values.

Correction to the handout: it refers to both Survived and survived in different places. Use the exact spelling shown by names(titanic). R is case-sensitive.

9. Visualization program: Bar chart

9.1 Bar chart - survivors by passenger class

Purpose: Compare passenger counts across classes and show survival status as separate, side-by-side bars.

Source alignment: Based on the handout's bar-chart code, with consistent column spelling and a theme added for readability.

Complete code

```
bar_chart <- ggplot(
  titanic,
  aes(x = factor(Pclass), fill = factor(Survived))
) +
  geom_bar(position = "dodge") +
  labs(
    title = "Survivors by Passenger Class",
    x = "Passenger Class",
    y = "Count",
    fill = "Survived"
  ) +
  scale_fill_manual(values = c("0" = "red", "1" = "blue")) +
  theme_minimal()

print(bar_chart)
```

Expected output / result

```
A bar chart appears in the RStudio Plots pane. The x-axis shows passenger class; the y-axis shows row counts. Exact bar heights de
```

Line-by-line explanation

- ggplot(titanic, aes(...)) starts the chart and maps columns to visual properties.
- factor(Pclass) treats the passenger classes as categories.
- fill = factor(Survived) uses different fill colours for survival categories.
- geom_bar(position = "dodge") counts records and places category bars side by side.
- labs() sets chart title, axis titles and legend title.
- scale_fill_manual() assigns red to 0 and blue to 1.
- theme_minimal() applies a clean theme.
- print(bar_chart) renders the chart.

Important checks

- If the plot errors, check whether Pclass and Survived exist with exactly that capitalization.

Viva questions

- Which package provides ggplot()? ggplot2.
- What does geom_bar() count? Rows in each category combination.
- What does dodge mean? Side-by-side bars rather than stacked bars.

10. Visualization program: Density plot

The uploaded handout labels this example “Line Plot - Average Fare by Age Group” but the supplied code actually uses geom_density() and plots age distribution, not average fare by age group. To avoid learning the wrong description, this guide names it correctly as a density plot. A density plot is a smoothed view of the distribution of a numeric variable.

10.1 Age distribution using a density plot

Purpose: Show the distribution of passenger ages as a smoothed curve.

Source alignment: Uses the plotting method shown on page 6 of the visualization handout; removes rows with missing ages for a robust plot.

Complete code

```
age_data <- subset(titanic, !is.na(Age))

line_plot <- ggplot(age_data, aes(x = Age)) +
  geom_density(fill = "blue", alpha = 0.5) +
  labs(
    title = "Age Distribution of Passengers",
    x = "Age",
    y = "Density"
  ) +
  theme_minimal()

print(line_plot)
```

Expected output / result

```
A smooth density curve appears. Peaks indicate ages with relatively higher estimated density. The plot is not a count of passenger
```

Line-by-line explanation

- subset(titanic, !is.na(Age)) excludes rows where Age is missing.
- aes(x = Age) places Age on the horizontal axis.
- geom_density() estimates and draws a smoothed distribution curve.
- fill = "blue" colours the area and alpha = 0.5 makes it semi-transparent.
- labs() sets the title and axis labels.
- print(line_plot) displays the result.

Important checks

- The handout’s title says average fare by age group, but the code does not compute average fare. A true average-fare-by-age-group line plot would require grouping ages and calculating mean Fare first.

Viva questions

- Which function draws a density curve? geom_density().
- What does NA mean? A missing value.
- What does alpha = 0.5 control? Transparency.

Optional: A true line plot of average fare by age group

Use this if an examiner specifically asks for average fare by age group. It is an additional corrected example, not the exact code printed in the handout.

```
titanic$AgeGroup <- cut(titanic$Age, breaks = c(0, 12, 18, 30, 45, 60, Inf), right = FALSE)

fare_by_age <- titanic %>%
  filter(!is.na(AgeGroup), !is.na(Fare)) %>%
  group_by(AgeGroup) %>%
  summarise(MeanFare = mean(Fare), .groups = "drop")

line_avg_fare <- ggplot(fare_by_age, aes(x = AgeGroup, y = MeanFare, group = 1)) +
  geom_line() + geom_point() +
  labs(title = "Average Fare by Age Group", x = "Age Group", y = "Mean Fare") +
  theme_minimal()

print(line_avg_fare)
```

Output: a line chart with age group on the x-axis and the mean fare of passengers in that group on the y-axis. Exact values depend on the dataset.

Line by line: cut() creates age bands; filter() removes rows with missing group/fare values; group_by() defines one group per age band; summarise() calculates mean fare; geom_line() draws connected means and geom_point() shows each group mean.

11. Visualization program: Scatter plot

11.1 Age versus fare, coloured by survival

Purpose: Explore the relationship between passenger age and fare; colour points by survival category.

Source alignment: Based on the handout's scatter plot, with missing age/fare rows excluded and consistent labels.

Complete code

```
scatter_data <- subset(titanic, !is.na(Age) & !is.na(Fare))

scatter_plot <- ggplot(
  scatter_data,
  aes(x = Age, y = Fare, color = factor(Survived))
) +
  geom_point() +
  labs(
    title = "Fare vs Age",
    x = "Age",
    y = "Fare",
    color = "Survived"
  ) +
  scale_color_manual(values = c("0" = "red", "1" = "steelblue")) +
  theme_minimal()

print(scatter_plot)
```

Expected output / result

```
A scatter plot appears. Each dot represents a passenger with non-missing age and fare. Red and blue indicate survival categories.
```

Line-by-line explanation

- subset() removes rows missing Age or Fare.
- aes(x = Age, y = Fare) defines horizontal and vertical numeric variables.
- color = factor(Survived) colours observations by survival category.
- geom_point() draws one point per row.
- scale_color_manual() sets colours for the two survival labels.
- The colour legend explains which survival category each colour represents.

Important checks

- Check actual survival values before using manual color labels 0 and 1.

Viva questions

- What is a scatter plot used for? To view the relationship between two numeric variables.
- What does each dot mean? One observation/passenger.
- Difference between fill and color? fill typically fills shapes/areas; color changes outlines/points/lines.

12. Visualization program: Heatmap

The handout introduces a heatmap using Pclass and Survived and stat_bin_2d(). The code below provides a stable version for two numeric-coded columns and excludes missing values. This is a frequency heatmap, not a calculated survival-rate heatmap.

12.1 Frequency heatmap: passenger class versus survival status

Purpose: Display how many passengers fall into each passenger-class and survival-status combination using colour intensity.

Source alignment: A runnable correction of the handout's heatmap example. The printed handout uses stat_bin_2d() and labels the fill as Frequency.

Complete code

```
heatmap_data <- subset(titanic, !is.na(Pclass) & !is.na(Survived))
heatmap_data$Survived_num <- as.numeric(as.character(heatmap_data$Survived))

heat_map <- ggplot(heatmap_data, aes(x = Pclass, y = Survived_num)) +
  stat_bin_2d(bins = 10, aes(fill = after_stat(count))) +
  labs(
    title = "Titanic Survival Frequency Heatmap",
    x = "Passenger Class",
    y = "Survived (0 = No, 1 = Yes)",
    fill = "Frequency"
  ) +
  scale_fill_gradient(low = "white", high = "blue") +
  theme_minimal()

print(heat_map)
```

Expected output / result

```
A heatmap appears, with colours representing counts in 2D bins. Exact counts depend on the CSV. For discrete passenger classes and
```

Line-by-line explanation

- subset() removes rows missing class or survival.
- as.numeric(as.character(Survived)) converts factor labels such as "0" and "1" back to numbers.
- aes(x = Pclass, y = Survived_num) sets the two axes.
- stat_bin_2d() counts records in two-dimensional bins.
- after_stat(count) maps the computed bin count to fill colour.
- scale_fill_gradient() maps low counts to white and high counts to blue.

Viva questions

- What does the colour intensity indicate? Frequency in this chart.
- What does stat_bin_2d() do? Counts observations in 2D bins.
- Is this a survival-rate heatmap? No, this one encodes frequency/count.

Alternative: a categorical frequency tile heatmap

For discrete passenger classes and survival categories, this version directly counts their combinations and draws a tile for each combination.

```
library(dplyr)

heat_counts <- titanic %>%
  filter(!is.na(Pclass), !is.na(Survived)) %>%
  count(Pclass, Survived)

heat_cat <- ggplot(heat_counts, aes(x = factor(Pclass), y = factor(Survived), fill = n)) +
  geom_tile(color = "white") +
  labs(title = "Passenger Count by Class and Survival",
       x = "Passenger Class", y = "Survived (0 = No, 1 = Yes)", fill = "Count") +
  scale_fill_gradient(low = "white", high = "blue") +
  theme_minimal()

print(heat_cat)
```

Output: a grid of tiles where each tile represents one class/survival combination. Darker blue indicates more passengers. This is often clearer for categorical combinations than 2D numeric binning.

13. Convert charts to interactive plots with plotly

The handout uses ggplotly() to convert ggplot2 charts to interactive visuals. In the RStudio Viewer pane or browser, hovering can show tooltips, dragging can zoom, and double-clicking can reset zoom.

```
interactive_bar_chart <- ggplotly(bar_chart)
interactive_bar_chart

interactive_line_plot <- ggplotly(line_plot)
interactive_line_plot

interactive_scatter_plot <- ggplotly(scatter_plot)
interactive_scatter_plot

interactive_heat_plot <- ggplotly(heat_map)
interactive_heat_plot
```

Explanation

- ggplotly(chart_object) converts a supported ggplot object into an interactive Plotly object.
- Writing the object name on its own displays it in an interactive-capable output pane.
- Tooltips, zoom and legend interactions depend on chart type and viewing environment.

Viva

- Which package provides ggplotly()? plotly.
- What does interactivity add? Hover details and controls such as zoom and legend toggling.
- Why might an interactive chart not appear in the Plots pane? It usually renders in the Viewer pane or browser.

14. Export static and interactive charts

14.1 Export a static chart

```
ggsave("bar_chart.png", plot = bar_chart, width = 8, height = 6)
ggsave("line_plot.jpg", plot = line_plot, width = 8, height = 6)
ggsave("scatter_plot.png", plot = scatter_plot, width = 8, height = 6)
ggsave("heat_map.png", plot = heat_map, width = 8, height = 6)
```

- ggsave() saves a plot to a file. The file extension determines the format, for example .png or .jpg.
- width and height set dimensions, normally in inches.
- Files are saved in the current working directory unless a path is specified.

14.2 Export an interactive chart to HTML

```
saveWidget(
  ggplotly(bar_chart),
  file = "interactive_bar_chart.html",
  selfcontained = TRUE
)
```

- saveWidget() comes from htmlwidgets. It saves an interactive widget as HTML.
- selfcontained = TRUE attempts to place required dependencies inside a single HTML file; this may require the webshot/zip-related dependencies and can fail in some setups. If it fails, use selfcontained = FALSE to save supporting files beside the HTML.

Viva

- ggsave() vs saveWidget()? ggsave exports static ggplot images; saveWidget exports interactive widgets as HTML.
- How do you choose file format in ggsave()? By filename extension.
- Where is the file saved? Usually in getwd(), the current working directory, unless an explicit path is used.

15. Troubleshooting and code corrections

Problem	Likely cause	Fix
object not found: Pclass	Column name differs or wrong CSV loaded	Run names(titanic), then use the exact column name.
object not found: survived	Case mismatch	Use Survived if that is the exact column name.
could not find function ggplot	ggplot2 not loaded/installed	Install if needed, then library(ggplot2).
could not find function ggplotly	plotly not loaded/installed	Install if needed, then library(plotly).
plot has no data / warnings about NA	Missing values or incompatible data	Use subset()/filter() to remove missing values needed by the chart.
manual colour scale warning	Labels in data do not match "0" and "1"	Inspect unique(titanic$Survived) and adjust scale values.
CSV file not found	Wrong path or working directory	Use file.choose() or set a correct absolute path.
HTML export fails	Self-contained dependency bundling issue	Try selfcontained = FALSE and check htmlwidgets dependencies.

Check the data before plotting

```
names(titanic)
str(titanic)
summary(titanic)
sum(is.na(titanic$Age))
sum(is.na(titanic$Fare))
unique(titanic$Survived)
```

Avoid blindly running code with an unfamiliar dataset. Different Titanic files can use different column names, rows and survival labels. The plotting calls are correct only when the data structure matches their assumptions.

16. Viva bank: short answers

Question	Short answer
What is R?	An open-source language/environment widely used for statistical computing, data analysis and graphics.
What is ggplot2?	An R package for building charts in layers.
What is a data frame?	A tabular structure whose columns can have different types, with equal-length columns.
What does read.csv() do?	Reads a CSV file into a data frame.
Why use str()?	To inspect columns, types and the structure of a data object.
What is aes()?	Maps data variables to visual properties such as axes, colour and fill.
What is geom_bar()?	Draws bars; by default it counts observations by category.
What is geom_point()?	Draws points, commonly used for scatter plots.
What is geom_density()?	Draws a smoothed estimate of a numeric variable distribution.
What is geom_tile()?	Draws rectangular tiles, useful for heatmaps.
What does labs() do?	Sets titles, axis labels and legend labels.
What does theme_minimal() do?	Applies a minimal chart theme.
What does factor() do?	Treats values as categorical levels.
What does ggplotly() do?	Converts a ggplot chart to an interactive Plotly chart.
What does ggsave() do?	Exports a static plot to an image/PDF format supported by its extension.
What does saveWidget() do?	Saves an HTML widget, including interactive Plotly charts.
What does dplyr::filter() do?	Keeps rows matching conditions.
What does group_by() do?	Defines groups for subsequent grouped operations.
What does summarise() do?	Calculates summary values such as means or counts.
What is a heatmap?	A chart that uses colour intensity to encode a value such as count or rate.

17. Quick revision checklist

- [ ] Can create vectors with c() and seq(), then filter with logical indexing.
- [ ] Can explain list indexing: [ ] keeps a list; [[ ]] extracts an element; $ accesses by name.
- [ ] Can create and index a matrix; knows the default fill is column-wise unless byrow = TRUE.
- [ ] Knows + and - for matrices, * element-wise multiplication, %*% matrix multiplication, and t() transpose.
- [ ] Can use colSums(), rowMeans(), sum() and apply(M, 2, sort).
- [ ] Can use readline(), nchar(), toupper(), substring() and integer division %/%.
- [ ] Can explain the palindrome function using strsplit(), rev() and paste(collapse = "").
- [ ] Can import and inspect CSV data with read.csv(), names(), str() and head().
- [ ] Can build bar, density, scatter and heatmap visualizations using the supplied dataset.
- [ ] Can explain interactivity via ggplotly() and export via ggsave() / saveWidget().
- [ ] Has checked exact column names and missing values before running visualization code.

Final practical strategy

- First run code in small sections and resolve any package or filename issue before building the chart.
- If the plot errors, check object names, column spelling, data types, missing values and packages before rewriting the whole program.
- For viva, describe what each major function does rather than memorizing an output that changes with the dataset.

Source basis: uploaded Data Analysis Using R Programming Lab syllabus/notebook materials and the “Data Visualization with R - Titanic Dataset, ggplot2 & plotly” handout. Exact chart values depend on the actual CSV file.